

2 TIER ARCHITECTURE PROJECT

VINEETH KUMAR M

INDEX PAGE

- 1. INTRODUCTION**
- 2. ARCHITECTURE**
- 3. WORKFLOW**
- 4. EXPLANATION**
- 5. RESULT**

1.INTRODUCTION

This project is a cloud-based student form creation developed using AWS services. It follows a multi-tier architecture where frontend, backend, and database are deployed separately.

2.ARCHITECTURE:

- a. Frontend – ec2 server (ubuntu) using html.
- b. Backend – flask api (python)

3.WORKFLOW:

- a. Create frontend EC2 server
- b. Create VPC for EC2
- c. Create backend EC2 server
- d. Create a mysql DB
- e. Install nginx server on frontend
- f. Install python on backend

4 – EXPLANATION:

Step-1 Create a VPC

Step-2 Create 2 subnets,

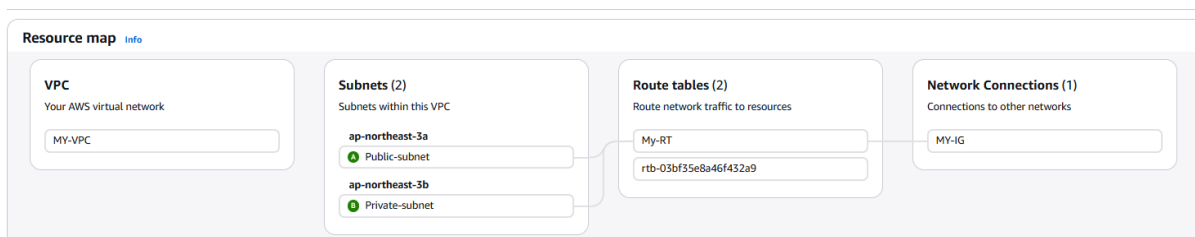
- Create 1 public subnet – frontend.
- Create 1 private subnet – Backend

Step-3 Create Route table,

- Create a Public RT.
- Create a Private RT.

Step-4 Create internet gateway then,

- Attach the IGW to VPC.
- Attach the public subnet to the public Route table.
- Attach the private subnet to the public Route table.
- Edit subnet association explicit connection.



Step -5 Create a EC2 INSTANCE

Instances (2) Info								less
Saved filter sets								
Choose filter set								
Find Instance by attribute or tag (case-sensitive)								
☐	Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	
☐	FRONTEND	i-06b21b97d21817cc2	Running	t3.micro	3/3 checks passed	View alarms +	ap-northeast-3a	
☐	BACKEND	i-036ac1d3400859e2a	Running	t3.micro	3/3 checks passed	View alarms +	ap-northeast-3b	

Step-6 Launch front-end EC2 instance

- Frontend – Nginx.
- Operating system: Ubuntu AMI
- Subnet: Public
- Attached the SG group
- Auto assign IP.

```

Welcome to Ubuntu 26.04 LTS (GNU/Linux 7.0.0-1004-aws x86_64)

 * Documentation:  https://docs.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Fri May 1 09:44:15 UTC 2026

System load:  0.08      Temperature:   -273.1 C
Usage of /:   34.0% of 6.61GB    Processes:    121
Memory usage: 31%      Users logged in: 0
Swap usage:   0%          IPv4 address for ens5: 10.0.2.142

Expanded Security Maintenance for Applications is not enabled.

3 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

Last login: Fri May 1 06:04:32 2026 from 15.168.105.162
ubuntu@ip-10-0-2-142:~$

```

Step-6 Launch Back-end EC2 instance.

- Front end – Flash API
- Operating system: Ubuntu AMI
- Subnet: Private
- Attached the SG group
- Auto assign IP.

```

Welcome to Ubuntu 26.04 LTS (GNU/Linux 7.0.0-1004-aws x86_64)

 * Documentation:  https://docs.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Fri May 1 10:06:05 UTC 2026

System load:  0.0      Temperature:   -273.1 C
Usage of /:   38.7% of 6.61GB    Processes:    115
Memory usage: 31%      Users logged in: 0
Swap usage:   0%          IPv4 address for ens5: 10.0.6.92

Expanded Security Maintenance for Applications is not enabled.

15 updates can be applied immediately.
10 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

Last login: Fri May 1 06:48:50 2026 from 15.168.105.161
ubuntu@ip-10-0-6-92:~$

```

Step-7 Go-to Security group inbound rules

- Allow the ports which you need.
- Allow custom port 5000 (for python)

Inbound rules (5)										
Search Manage tags										
☐	Name	Security group rule ID	IP version	Type	Protocol	Port range	Source			
☐	-	sgr-054a6d20ecf5e7e7d	IPv4	HTTP	TCP	80	0.0.0.0/0			
☐	-	sgr-02566732b0993d6d1	IPv4	SSH	TCP	22	0.0.0.0/0			
☐	-	sgr-0551fec959441d889	IPv4	All ICMP - IPv4	ICMP	All	0.0.0.0/0			
☐	-	sgr-0d1d76c850d0ad5b6	IPv4	HTTPS	TCP	443	0.0.0.0/0			
☐	-	sgr-0d5298c5d3b0ef568	IPv4	Custom TCP	TCP	5000	0.0.0.0/0			

Connect your server and use below some commands to update your kernel and install nginx,

LINUX – OPERATING SYSTEM

- `sudo apt update -y`

```
ubuntu@ip-10-0-6-92:~$ sudo apt update -y
```

- `sudo nano "key pair name"` (update your key pair name)

```
ubuntu@ip-10-0-2-142:~$  
ubuntu@ip-10-0-2-142:~$ sudo nano "key pair name"
```

- `sudo chmod 400 "key pair name"` (change the permission to your key pair)

```
ubuntu@ip-10-0-6-92:~$  
ubuntu@ip-10-0-6-92:~$ sudo chmod 400 "key pair name"
```

NGINX - SERVER

Install nginx server in frontend by using below commands:

- `sudo apt install nginx`
- default path (nginx) `var/www/html/index.nginx-debian.html`
- update your html code in that .html page
- now your front-end is working

Create a connection between FRONT-END TO BACKEND



FRONT-END ===== API ===== BACK-END

BACKEND - SERVER

Connect your server and use below some commands to install nginx and update your kernel,

- `sudo apt update -y`

```
ubuntu@ip-10-0-6-92:~$ sudo apt update -y
```

- `sudo nano "key pair name"` (update your key pair name)

```
ubuntu@ip-10-0-2-142:~$  
ubuntu@ip-10-0-2-142:~$ sudo nano "key pair name"
```

- `sudo chmod 400 "key pair name"` (change the permission to key pair)

```
ubuntu@ip-10-0-6-92:~$  
ubuntu@ip-10-0-6-92:~$ sudo chmod 400 "key pair name"
```

You need to install python in you back-end-server by using below commands,

- `sudo apt install python3-pip-y`

```
ubuntu@ip-10-0-2-142:~$  
ubuntu@ip-10-0-2-142:~$ sudo apt install python3-pip -y
```

- `pip3 install flask flask-cors --break-system-packages`

```
ubuntu@ip-10-0-2-142:~$  
ubuntu@ip-10-0-2-142:~$  
ubuntu@ip-10-0-2-142:~$ pip3 install flask flask-cors --break-system-packages
```

- create app.py (update your flask code)

```
ubuntu@ip-10-0-2-142:~$  
ubuntu@ip-10-0-2-142:~$ sudo nano app.py
```

- `python3 app.py` (now python is running on your server)

```
ubuntu@ip-10-0-2-142:~$  
ubuntu@ip-10-0-2-142:~$  
ubuntu@ip-10-0-2-142:~$ sudo apt install python3-pip -y
```

Use the below html code in front-end-server

- go to your nginx default path and,
- edit the .html file with below html

```
<!DOCTYPE html>  
    <!DOCTYPE html>  
<html>  
<head>  
    <title>Student Registration</title>  
    <style>  
        body {  
            font-family: Arial;  
            background: #f2f2f2;  
            padding: 30px;  
        }  
    }
```

```
.box {
  width: 320px;
  background: white;
  padding: 20px;
  border-radius: 10px;
  box-shadow: 0 0 10px gray;
}

input, button {
  width: 100%;
  margin-top: 10px;
  padding: 8px;
}

button {
  background: blue;
  color: white;
  border: none;
  cursor: pointer;
}

#list {
  margin-top: 20px;
  padding: 10px;
  background: #e6ffe6;
  border-radius: 5px;
}
</style>
</head>
<body>

<div class="box">
  <h2> Student Form</h2>

  <form id="form">
    <input type="text" id="name" placeholder="Name" required>
    <input type="number" id="age" placeholder="Age" required>
    <input type="text" id="course" placeholder="Course" required>

    <button type="submit">Submit</button>
```



```
</form>
```

```
<div id="list"></div>
</div>
```

```
<script>
```

```
const listDiv = document.getElementById("list");
```

```
document.getElementById("form").addEventListener("submit", function(e) {
  e.preventDefault();
```

```
const data = {
  name: document.getElementById("name").value,
  age: document.getElementById("age").value,
  course: document.getElementById("course").value
};
```

```
fetch("http://YOUR_IP_ADDRESS:5000/student", {
  method: "POST",
  headers: {
    "Content-Type": "application/json"
  },
  body: JSON.stringify(data)
})
```

```
.then(res => res.json())
.then(response => {
  listDiv.innerHTML = "<h3>Student List:</h3>";
```

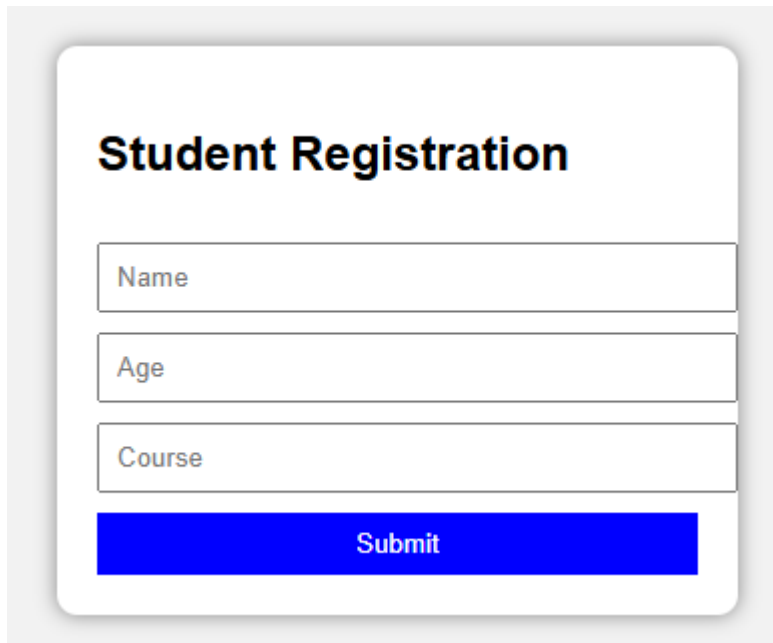
```
  response.students.forEach((s, index) => {
    listDiv.innerHTML += `
      <p>${index + 1}. ${s.name} - Age: ${s.age} - Course: ${s.course}</p>
    `;
  });
})
```

```
.catch(error => {
  listDiv.innerHTML = "✖ Server Error!";
  console.error(error);
});
});
</script>
```

</body>

</html>

- Try your Frontend ip address in your chrome.



The image shows a web form titled "Student Registration". It contains three input fields: "Name", "Age", and "Course". Below these fields is a blue "Submit" button. The form is styled with a white background and rounded corners, set against a light gray background.

- Go to backend server and update your app.py by using below code which you created,

Update your app.py (flask code) by below code: **Just copy paste**

```
from flask import Flask, request, jsonify
from flask_cors import CORS
```

```
app = Flask(__name__)
CORS(app)
```

```
students = [] # store all students
```

```
@app.route('/student', methods=['POST'])
```

```
def add_student():
```

```
    data = request.get_json()
```

```
    student = {
```

```
        "name": data.get('name'),
```

```
        "age": data.get('age'),
```

```
        "course": data.get('course')
```

```
    }
```

```
students.append(student)
```

```
return jsonify({  
    "message": "Student added successfully",  
    "students": students  
})
```

```
if __name__ == '__main__':  
    app.run(host='0.0.0.0', port=5000, debug=True)
```

The screenshot displays a web interface for 'Student Registration'. It features three input fields containing the text 'Mohan', '28', and 'AWS'. Below these fields is a blue 'Submit' button. At the bottom, a light green message box states 'Student added successfully'.

Student Registration	
Mohan	
28	
AWS	
Submit	
Student added successfully	

THANK YOU